Project 12: Jenkins Pipeline for MyMavenWebApp

Objective

The goal of this project is to create a Jenkins Declarative Pipeline job for MyMavenWebApp that automates the process of checking out the source code from GitHub, building the Maven project, archiving the generated WAR artifact, and deploying it. The pipeline is defined using a Jenkinsfile committed to the GitHub repository and pulled into Jenkins via Pipeline script from SCM.

Steps

Step 1: Create and Push the Jenkinsfile to GitHub

Open the terminal, navigate into the MyMavenWebApp project folder, create a Jenkinsfile containing the pipeline code shown below, then commit and push it to the GitHub repository.

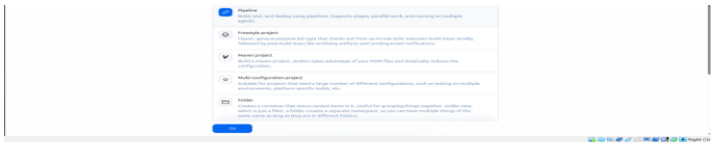
```
pipeline {
    agent any
    tools {
        maven 'Maven'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main2', url:
'https://github.com/Someesvaar/MyMavenWebApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Archive') {
            steps {
                archiveArtifacts artifacts: 'target/*.war', fingerprint: true
            }
        }
        stage('Deploy') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
    post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

The pipeline defines four stages: Checkout pulls the source from the main2 branch, Build compiles and packages the project using Maven, Archive saves the generated WAR file as a

build artifact with fingerprinting enabled, and Deploy re-runs the Maven package goal to place the artifact. Post-build actions echo the result of the build.

Step 2: Create a New Jenkins Pipeline Job

Go to the Jenkins home page and click New Item. Enter the item name as MyMavenWebApp and select Pipeline as the project type. This option allows building, testing, and deploying applications using a Jenkinsfile. Click OK to proceed to the job configuration page.



Step 3: In **MyMavenWebApp**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenWebApp**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MyMavenWebApp folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.

Fig 1: Jenkins New Item page — selecting Pipeline as the project type

Step 3: Configure the Pipeline — Pipeline Script from SCM

In the MyMavenWebApp job configuration, navigate to the Pipeline section. Under Definition, select Pipeline script from SCM. Set SCM to Git and enter the Repository URL of the MyMavenWebApp GitHub repository. In the Credentials field, select the previously saved GitHub credentials. Leave the Script Path as Jenkinsfile (the default). Click Apply and Save to finalise the configuration.

- **`Definition:`** Pipeline script from SCM
- **`SCM:`** Git
- **`Repository URL:`** https://github.com/Someesvaar/MyMavenWebApp.git
- **`Credentials:`** Github Credintials
- **`Script Path:`** Jenkinsfile

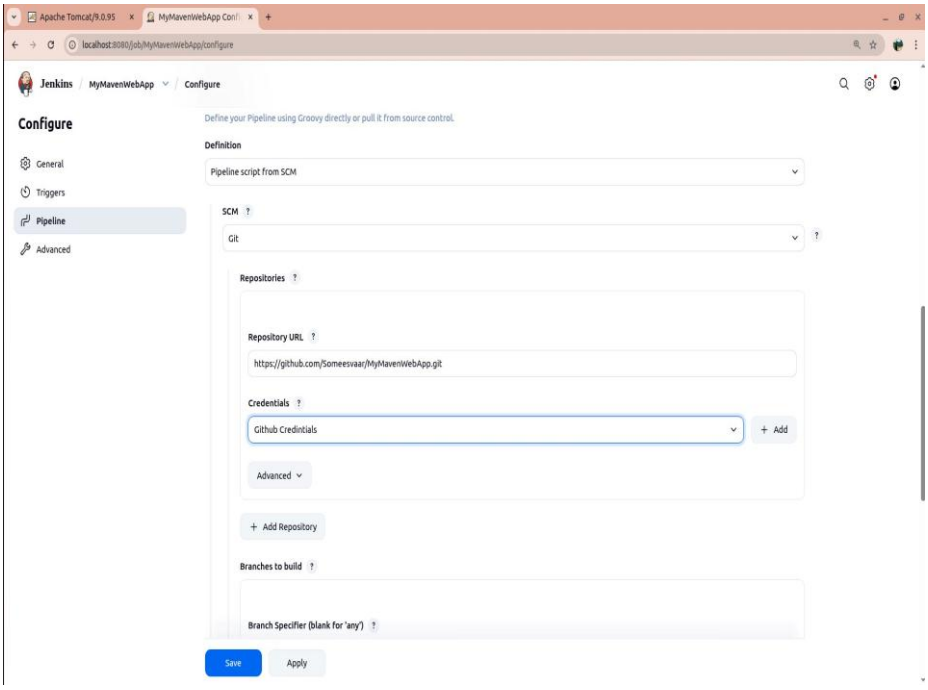


Fig 2: Jenkins pipeline configuration showing Git SCM with GitHub URL and credentials

Step 4: Run the Pipeline — Build Now

After saving the configuration, return to the MyMavenWebApp job page and click Build Now. Jenkins fetches the Jenkinsfile from the GitHub repository, installs the required Maven tool, and executes each stage in sequence. The Stage View confirms that build #3 completed successfully with all stages passing.

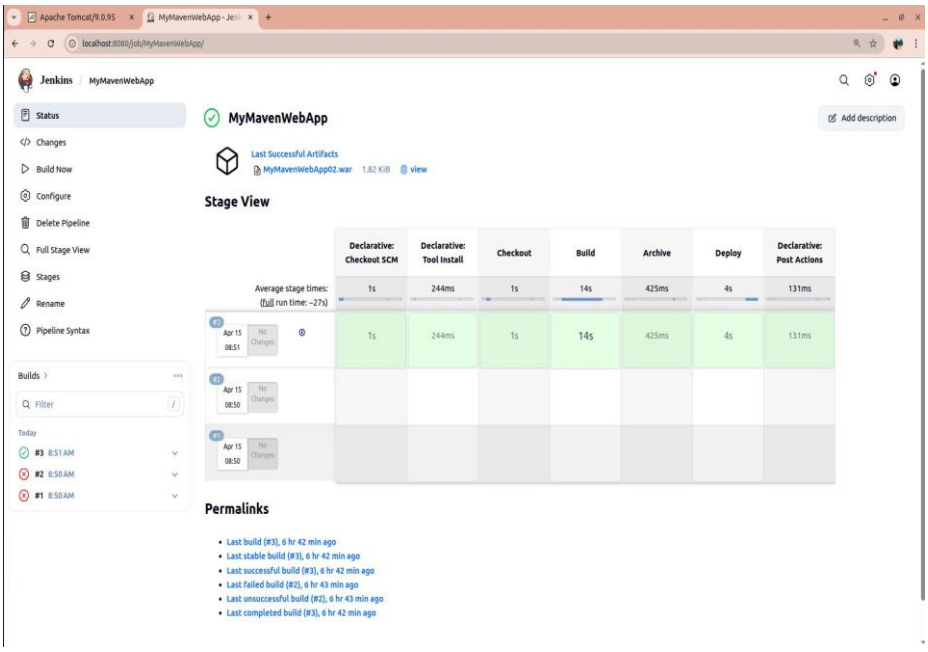


Fig 3: Jenkins Stage View for MyMavenWebApp — Build #3 successful with all stages green and MyMavenWebApp02.war artifact archived (1.82 KiB)